

```

1 %=====
2 % Lucas Tree Model
3 %=====
4
5 clear
6 clc
7
8 gamma=1.0;      %Coefficient of Constant Risk Aversion
9 beta=0.95;      %Discount Factor
10 alpha=0.9;      %\log y_{t+1}= \alpha \log y_t + \sigma \varepsilon_t
11 sigma=0.1;      %Standard deviation for the Log-Normal Distribution
12
13 %Grid for y
14 %Since we cannot just have a state space [0,\infty), we consider [y_min, y_max]
15 ssd=sigma/sqrt(1-alpha^2);      %The standard deviation for \log y_t
16 y_min=exp(-4*ssd);      %The minimum of the grid
17 y_max=exp(4*ssd);      %The maximum of the grid
18 dim_y=100;      %Grid size for the vector of y
19 y_vec=linspace(y_min,y_max,dim_y); %Grid for y
20
21 %Realizations of y_t (to compute/approximate integrals)
22 %Two equivalent ways
23 %epsilon: (1,500)-vector, iid standard normal
24 %r_ln: (1,500)-vector, iid log-normal with (mean,sd)=(0,sigma)
25 rng('default');      %Initialize the random number generator
26 epsilon=normrnd(0,1,1,500); %Statistics and Machine Learning Toolbox
27 rng('default');      %Initialize the random number generator
28 r_ln=lognrnd(0,sigma,1,500); %Statistics and Machine Learning Toolbox
29 %Note: r_ln=exp(sigma*epsilon);
30 %      y_{t+1} = y^{\alpha}_t \exp ( \sigma \varepsilon_t )
31
32 %h(y) = beta* \int u'(G(y,z)) G(y,z) \phi(dz)
33 h=zeros(1,dim_y);
34 for i=1:dim_y
35     h(i) = beta* mean(((y_vec(i)^alpha).*r_ln).^(1-gamma));
36 end
37
38 f=zeros(1,dim_y);      %f
39 f_new=zeros(1,dim_y);  %Tf
40
41 max_iter=2000;      %The Maximum Number of Iterations
42 toler=0.1^4;      %Tolerance Value
43 converge=0;
44 % converge is a binary variable, where it takes 1 when the value
45 % function iteration ended within the maximum number of rounds (max_iter).
46
47 tic %start stopwatch timer (optional)
48 for ell=1:max_iter
49     for i=1:dim_y
50         f_new(i) = h(i) + beta* mean( interp1(y_vec,f,(y_vec(i)^alpha).*r_ln,'spline') );
51     end %end of for i=1:dim_N
52     %
53     if norm(f_new-f,Inf)<toler %alternative: if max(abs(f_new-f))<toler
54         converge=1;
55         break;
56     end %end of if
57     %
58     f=f_new;

```

```

59 end
60 comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)
61
62 price=f.*y_vec.^gamma;
63
64 %Convergence
65 if converge==1
66     fprintf('The value function iteration converged with %d iterations.\n', ell);
67     fprintf('The value function iteration took %f seconds.\n', comp_time);
68 else
69     fprintf('The value function iteration did not converge.\n');
70 end
71
72 %Plot
73 figure()
74 hold on
75 plot(y_vec,price,'LineWidth',2);
76 if gamma==1
77     plot(y_vec,(beta/(1-beta))*y_vec,'--','LineWidth',2);
78 end
79 grid on
80 hold off
81 xlabel('$y$', 'Interpreter', 'latex', 'FontSize', 12);
82 ylabel('$p$', 'Interpreter', 'latex', 'FontSize', 12);
83 title('Equilibrium Pricing Function $p$', 'Interpreter', 'latex', 'FontSize', 14);
84 xlim([y_min, y_max]);
85

```